

CKA - Jun 2026 - Part 2

Part1 - https://miro.com/app/board/uXjVHDOXyJ0=

SECRETS

- 1 kubectl get secrets
- 2 echo -n "admin" | base64 > username.txt
- 3 echo -n "password123" | base64 > password.txt
- 4 cat username.txt
- 5 cat password.txt
- 6 kubectl create secret generic db-user-pass --from-file=./username.txt --from-file=./password.txt
- 7 kubectl get secrets
- 8 kubectl get secrets
- 9 kubectl describe secrets db-user-pass
- 10 history

```
---
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data :
  username: YWRtaW4= # base64 encoded value of 'admin'
  password: cGFzc3dvcmQxMjM= # base64 encoded value of 'password123'
```

```

---

apiVersion : v1
kind : Pod
metadata :
  name : mysecretpod
  labels :
    name : mysecretpod
spec:
  containers :
    - name : mysecretpod
      image : redis
      volumeMounts :
        - name : mysecretvolume
          mountPath : "/etc/secret-volume"
          readOnly : true

  volumes :
    - name : mysecretvolume
      secret :
        secretName : my-secret
---

apiVersion : v1
kind : Pod
metadata :
  name : secret-env-pod
spec :
  containers :
    - name : mycontainer
      image : redis
      envFrom :
        - secretRef :
            name : my-secret
  restartPolicy : Never

```

```

---

apiVersion : v1
kind : Pod
metadata :
  name : secret-env-pod-2
spec :
  containers :
    - name : mycontainer
      image : redis
      env :
        - name : SECRET_USERNAME
          valueFrom :
            secretKeyRef :
              name : my-secret
              key : username

        - name : SECRET_PASSWORD
          valueFrom :
            secretKeyRef :
              name : my-secret
              key : password
      restartPolicy : Never

```

- 14 cat > 20.1.yaml
- 15 kubectl create -f 20.1.yaml
- 16 kubectl get secrets
- 17 cat > 20.2.yaml
- 18 kubectl create -f 20.2.yaml
- 19 kubectl get po
- 20 kubectl exec -it mysecretpod -- /bin/bash

```
21 cat > 20.3.yaml
22 kubectl create -f 20.3.yaml
23 kubectl exec -it secret-env-pod -- /bin/bash
24 cat > 20.4.yaml
25 kubectl create -f 20.4.yaml
26 kubectl exec -it secret-env-pod-2 -- /bin/bash
27 history
```

CONFIG MAP

```
1 kubectl create configmap my-config-map --from-literal=key1=value1 --from-
literal=key2=value2
2 kubectl create configmap my-config-map1 --from-file=./application1.properties
--from-file=./application2.properties
3 cat > application1.properties
4 cat > application2.properties
5 kubectl create configmap my-config-map1 --from-file=./application1.properties
--from-file=./application2.properties
6 kubectl get cm
7 cat > 21.1.yml
8 cat > 21.2.yml
9 vi 21.2.yaml
10 vi 21.2.yml
11 kubectl create -f 21.1.yml -f 21.2.yml
12 cat > 21.3.yaml
13 cat > 21.4.yaml
14 cat > 21.5.yaml
15 kubectl create -f 21.3.yaml -f 21.4.yaml -f 21.5.yaml
16 kubectl get po
17 kubectl exec -it my-config-pod1 -- /bin/sh
18 kubectl exec -it my-config-pod2 -- /bin/sh
19 kubectl exec -it my-config-pod3 -- /bin/sh
20 history
```

```
application1.properties
```

```
management.endpoint.health.show-details=always  
management.endpoints.web.exposure.include=*  
management.endpoint.health.enabled=true  
management.endpoint.health.probes.enabled=true
```

```
application2.properties
```

```
server.port=8081  
spring.application.name=application2  
server.servlet.context-path=/application2  
url=http://localhost:8081/application2
```

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: my-config1  
data :  
  APP_NAME: "MyApp_Kubernetes"  
  ENVIRONMENT: "Production"  
  PORT: "8080"
```

```
- - -
```

```
apiVersion: v1  
kind: ConfigMap  
metadata:
```

```

name: my-config2

data :
  db_url : "jdbc:mysql://localhost:3306/mydb"
  db_host : "localhost"
  db_port : "3306"

  service-details : |
    name: "MyService"
    version: "1.0.0"
    description: "This is a sample service configuration."

  css-properties : |
    body {
      background-color: #f0f0f0;
      font-family: Arial, sans-serif;
    }
    h1 {
      color: #333333;
    }

---

apiVersion : v1
kind : Pod
metadata :
  name : my-config-pod1
spec :
  containers :
    - name : my-container1
      image : busybox
      command : ["sh", "-c", "echo $APP_NAME && echo $ENVIRON
MENT && echo $PORT && sleep 3600"]
      envFrom :
        - configMapRef :

```

```

        name : my-config1

---

apiVersion : v1
kind : Pod
metadata :
  name : my-config-pod2
spec :
  containers :
    - name : my-container2
      image : busybox
      command : ["sh", "-c", "echo $APP_NAME && echo $ENVIRON
MENT && echo $PORT && sleep 3600"]
      env :
        - name : APP_NAME
          valueFrom :
            configMapKeyRef :
              name : my-config1
              key : APP_NAME
        - name : ENVIRONMENT
          valueFrom :
            configMapKeyRef :
              name : my-config1
              key : ENVIRONMENT
        - name : PORT
          valueFrom :
            configMapKeyRef :
              name : my-config1
              key : PORT

---

apiVersion : v1
kind : Pod

```

```

metadata :
  name : my-config-pod3
  labels :
    app : my-config-pod3
spec :
  containers :
    - name : my-container3
      image : redis
      volumeMounts :
        - name : config-volume
          mountPath : /etc/config

  volumes :
    - name : config-volume
      configMap :
        name : my-config2

---

```

```

apiVersion : v1
kind : Pod
metadata :
  name : my-config-pod4
spec :
  containers :
    - name : my-container4
      image : nginx:latest
      volumeMounts :
        - name : config-volume
          mountPath : /etc/config

  volumes :

```

```

- name : config-volume
  configMap :
    name : my-config-map1
    items :
      - key : application1.properties
        path : application1.properties

      - key : application2.properties
        path : application2.properties

---

```

NAMESPACES

```

cat > 22.yaml
2 kubectl create -f 22.yaml
3 kubectl get po
4 kubectl get po -n my-ns1
5 kubectl get po --namespace=my-ns1
6 kubectl run nginx --image=nginx
7 kubectl get po
8 kubectl get po --namespace=default
9 kubectl config set-context --current --namespace=my-ns1
10 kubectl get po
11 kubectl api-resources
12 kubectl config view --minify --output 'jsonpath={..namespace}'
13 history

```

```

---
apiVersion : v1
kind : Namespace
metadata :

```

```

    name : my-ns1

---

apiVersion : v1
kind : Namespace
metadata :
  name : my-ns2

---

apiVersion : v1
kind : Pod
metadata :
  name : my-config-pod1
  labels :
    app : my-config-pod1
  namespace : my-ns1

spec :
  containers :
    - name : my-container1
      image : nginx:latest

---

```

RESOURCE QUOTA

Create a resource quota named `app` under the namespace `rq-demo` using the following YAML definition in the file `resourcequota.yaml`.

```

apiVersion: v1
kind: ResourceQuota
metadata:

```

```
name: app
spec:
  hard:
    pods: "2"
    requests.cpu: "2"
    requests.memory: 500Mi
```

1. Create a new Pod that exceeds the limits of the resource quota requirements e.g. by defining 1Gi of memory but stays below the CPU e.g. 0.5. Write down the error message.
2. Change the request limits to fulfill the requirements to ensure that the Pod could be created successfully. Write down the output of the command that renders the used amount of resources for the namespace.

```
---

apiVersion : v1
kind : ResourceQuota
metadata :
  name : app
spec :
  hard :
    pods : "2"
    requests.cpu : "2"
    requests.memory : 500Mi
```

```
---

apiVersion : v1
kind : Pod
metadata :
  name : mypod
spec :
  containers :
    - name : mypod
```

```

    image : nginx:latest
    resources :
      requests :
        memory : "1Gi"
        cpu : "0.5"

---

apiVersion : v1
kind : Pod
metadata :
  name : mypod
spec :
  containers :
    - name : mypod
      image : nginx:latest
      resources :
        requests :
          memory : "255Mi"
          cpu : "0.5"

---

```

- 1 cat > 23.yaml
- 2 kubectl describe quota --namespace=default
- 3 kubectl create ns rq-demo
- 4 kubectl describe quota --namespace=rq-demo
- 5 kubectl apply -f 23.yaml
- 6 kubectl delete resourcequota rq-demo
- 7 kubectl delete resourcequota app
- 8 kubectl describe quota --namespace=default
- 9 kubectl apply -f 23.yaml --namespace=rq-demo

```
10 kubectl describe quota --namespace=rq-demo
11 cat > 23.1.yaml
12 kubectl apply -f 23.1.yaml --namespace=rq-demo
13 cat > 23.1.yaml
14 kubectl apply -f 23.1.yaml --namespace=rq-demo
15 kubectl describe quota --namespace=rq-demo
16 history
```

RESOURCE REQUESTS AND LIMITS

1. Define a Pod named `hello-world` running the container image `bmuschko/nodejs-hello-world:1.0.0`. The container exposes the port 3000.
2. Add a Volume of type `emptyDir` and mount it the container path `/var/log`.
3. For the container, specify the following minimum number of resources as follows:
 - CPU: 100m
 - Memory: 500Mi
 - Ephemeral storage: 1Gi
4. For the container, specify the following maximum number of resources as follows:
 - Memory: 500Mi
 - Ephemeral storage: 2Gi
5. Create the Pod from the YAML manifest.
6. Inspect the Pod details. Which node does the Pod run on?

```

apiVersion : v1
kind : Pod
metadata :
  name : hello
spec :
  containers :
    - name : hello
      image : bmuschko/nodejs-hello-world:1.0.0
      ports :
        - name : nodejs-port
          containerPort : 3000
      volumeMounts :
        - name : log-volume
          mountPath : /var/log
      resources :
        requests :
          memory : 500Mi
          cpu : 100m
          ephemeral-storage : 1Gi
        limits :
          memory : 500Mi
          ephemeral-storage : 2Gi

  volumes :
    - name : log-volume
      emptyDir : {}

```

```

17 cat > 24.yaml
18 kubectl apply -f 24.yaml
19 cat > 24.yaml
20 kubectl apply -f 24.yaml
21 kubectl get nodes
22 kubectl get po -o wide

```

- 24 kubectl describe pod hello
- 25 history

LIMIT RANGE

1. Inspect the YAML manifest definition in the file `setup.yaml`.
2. Create the objects from the YAML manifest file.
3. Create a new Pod named `pod-without-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` without any resource requirements. Inspect the Pod details. What resource definitions do you expect to be assigned?
4. Create a new Pod named `pod-with-more-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 400m and limits of 1.5. What runtime behavior do you expect to see?
5. Create a new Pod named `pod-with-less-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 350m and limits of 400m. What runtime behavior do you expect to see?

```
apiVersion: v1
kind: Namespace
metadata:
  name: d92
---
apiVersion: v1
kind: LimitRange
metadata:
```

```
name: cpu-limit-range
namespace: d92
spec:
  limits:
  - min:
      cpu: 200m
    max:
      cpu: 500m
    type: Container
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-without-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-more-cpu-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
    resources:
      requests:
        cpu: 400m
```

```
limits:
  cpu: 1.5
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-less-cpu-resource-requirements
  namespace: d92
spec:
  containers:
  - image: nginx:1.23.4-alpine
    name: nginx
    resources:
      requests:
        cpu: 350m
      limits:
        cpu: 400m
```

- 1 cat > 25.1.yaml
- 2 kubectl create -f 25.1.yaml
- 3 cat > 25.2.yaml
- 4 kubectl create -f 25.2.yaml
- 5 cat > 25.3.yaml
- 6 kubectl create -f 25.3.yaml
- 7 cat > 25.4.yaml
- 8 kubectl create -f 25.4.yaml
- 9 history

INGRESS

1. Verify that the Ingress Controller is running.

2. Create a new Deployment named `web` that controls a single replica running the image `bmuschko/nodejs-hello-world:1.0.0` on port 3000.
3. Expose the Deployment with a Service named `web` of type `ClusterIP`. The Service routes traffic to the Pods controlled by the Deployment `web`.
4. Make a request to the endpoint of the application on the context path `/`. You should see the message "Hello World".
5. Create an Ingress that exposes the path `/` for the host `hello-world.exposed`. The traffic should be routed to the Service created earlier.
6. List the Ingress object.
7. Add an entry in `/etc/hosts` that maps the virtual node IP address to the host `hello-world.exposed`.
8. Make a request to `http://hello-world.exposed`. You should see the message "Hello World".

```
1 kubectl create deployment web --image=bmuschko/nodejs-hello-world:1.0.0
2 kubectl get deploy
3 kubectl expose deployment web --type=ClusterIP --port=3000
4 kubectl get svc
5 kubectl get deploy
6 kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm -- wget -O-
web:3000
7 cat > 26.yaml
8 kubectl create -f 26.yaml
9 kubectl get ingress hello-world-ingress
10 history
```

```

apiVersion : networking.k8s.io/v1
kind : Ingress
metadata :
  name : hello-world-ingress
  annotations :
    nginx.ingress.kubernetes.io/rewrite-target : /$1
spec :
  rules :
    - host : hello-world.exposed
      http :
        paths :
          - path : /
            pathType : Prefix
            backend :
              service :
                name : web
                port :
                  number : 3000

```

```

$ kubectl get ingress hello-world-ingress

```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
hello-world-ingress	nginx	hello-world.exposed	192.168.64.38	80	72s

Accessing the Ingress

Accessing the Ingress differs depending on the Kubernetes cluster you are using. Follow the instructions in the section based on your Kubernetes cluster setup.

Using a Regular Kubernetes Cluster

Edit the file `/etc/hosts` via `sudo vim /etc/hosts`. Add the following entry to map the host name `hello-world.exposed` to the load balancer's IP address.

```
192.168.64.38 hello-world.exposed
```

Make a `curl` call to the host name mapped by the Ingress. The call should be routed toward the backend and respond with the message "Hello World".

```
$ curl hello-world.exposed
Hello World
```

Create a user "nec-adm". Grant nec-adm access to cluster, should have permissions to create, list, get, update, and delete pods in nec namespace
Private key exist in location: /vagrant/nec-adm.key and csr

```
---
```

```
apiVersion : certificates.k8s.io/v1
kind : CertificateSigningRequest
metadata :
  name : nec-adm
spec :
  request : LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBSRVFVRVNULS0tLS0KT
U1JQz1EQ0NBZHdDQVFBD2ZERUxNQWtHQTFVRUJoTUNTUVR4Q3pBSkNlZCQW
dNQWt0Qk1SSXdfQVlEVlFRSAPeQWxDWVc1bl1XeHZjbV44RERBS0JnTlZCQW9
NQTBWRV6RUxNQWtHQTFVRUN3d0NRV2t4RERBS0JnTlZCQU1NCkEwSlVakVq
TUNFR0NTcUdTSWIZRFFFSkFSWVVKbWx6YUhsdFlXbHNjMEJhYldGcGJDNWpiM
jB3Z2dFaU1BMEcKQ1NxR1NJYjNEUUVCCQVFVQUE0SUJEd0F3Z2dFS0FvSUJB
UU4MmpyYlllVbmNvcnR6Yn1ZaDVWdEZ3RUMrd3g0cQpxQ1NWN1RXYY0hmQWtuM2J
DWExTz1RpeE9jUmk4VVBWcXJjWwY3dHN2bFBBDTGpDVUtRd0drWnluNmF0VlBI
WisxClFCS2F4TFQ1dU9weDhCS0g0OTkxVWdCNzRxTG1zbFZId0JIT0FVTn1CY
mFuQW9GMm1rdEFNdnV3WW5mcH1NUWYKV2MxYkV0K3hCdWJ4TkE5SEpMSFVlQV
```



```

metadata :
  name : admin-pods
  namespace : nec
subjects :
  - kind : User
    name : nec-adm
    apiGroup : rbac.authorization.k8s.io
roleRef :
  kind : Role
  name : pod-admin
  apiGroup : rbac.authorization.k8s.io

---

```

- 1 kubectl create ns nec
- 2 openssl genrsa -out nec_adm.key 2048
- 3 ls
- 4 openssl req -new -key nec_adm.key -out nec-adm.csr
- 5 ls
- 6 cat nec-adm.csr | base64 | tr -d "\n"
- 7 cat > 27.1.yaml
- 8 kubectl create -f 27.1.yaml
- 9 kubectl get csr
- 10 kubectl certificate approve nec-adm
- 11 kubectl get csr
- 12 kubectl create -f 27.2.yaml
- 13 cat > 27.21.yaml
- 14 cat > 27.2.yaml
- 15 cat > 27.3.yaml
- 16 kubectl create -f 27.1.yaml -f 27.2.yaml
- 17 kubectl create -f 27.3.yaml
- 18 kubectl get roles -n nec
- 19 kubectl get rolebinding -n nec

```
20 kubectl get pods --as nec-adm -n default
21 kubectl get pods --as nec-adm -n nec
22 kubectl auth can-i get pods -n nec --as nec-adm
23 kubectl auth can-i get pods -n default --as nec-adm
24 history
```

CRD

1. Create a CRD resource named `backup.example.com` with the following specification:
 - Group: `example.com`
 - Version: `v1`
 - Kind: `Backup`
 - Singular: `backup`
 - Plural: `backups`
 - Properties of type `string`: `cronExpression`, `podName`, `path`
2. Retrieve the details for the `Backup` custom resource created in the previous step.
3. Create a custom object named `nginx-backup` for the CRD. Provide the following property values:
 - `cronExpression`: `0 0 * * *`
 - `podName`: `nginx`
 - `path`: `/usr/local/nginx`
4. Retrieve the details for the `nginx-backup` object created in the previous step.

```

---

apiVersion : apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.example.com
spec:
  group: example.com
  versions :
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                cronExpression :
                  type : string
                podName :
                  type : string
                path :
                  type : string

  scope : Namespaced
  names :
    kind : Backup
    singular : backup
    plural : backups
    shortNames :

```

```
- bu

---

apiVersion : example.com/v1
kind : Backup
metadata :
  name : nginx-backup
spec :
  cronExpression : "0 0 * * *"
  podName : nginx
  path : /usr/local/nginx

---
```

```
cat > 28.1.yaml
2 kubectl create -f 28.1.yaml
3 kubectl get crd
4 kubectl api-resources
5 cat > 28.2.yaml
6 kubectl create -f 28.2.yaml
7 kubectl get bu
8 kubectl get backups
9 kubectl get backup
10 kubectl describe backup nginx-backup
11 history
```

Upgrade given cluster (Master and worker node) from 1.35.1-00 to 1.35.6 / 1.35.0-00. Make sure to first drain respective node prior to update and make it available post update.

CONTROL PLANE

```
alias g=kubectl
2 g get nodes
3 sudo apt update
4 g drain controlplane --ignore-daemonsets
5 g get nodes
6 apt-cache madison kubeadm | head
7 sudo apt install kubeadm=1.35.6-1.1
8 sudo kubeadm upgrade apply v1.35.6
9 sudo apt install kubelet=1.35.6-1.1
10 sudo apt install kubelet=1.35.6-1.1
11 sudo systemctl restart kubelet
12 g uncordon controlplane
13 g get nodes
14 history
```

NODE 01

```
ssh node01
```

```
sudo apt update
2 sudo apt install kubeadm=1.35.6-1.1
3 sudo kubeadm upgrade node
4 sudo apt install kubelet=1.35.6-1.1
```

```
5 sudo systemctl restart kubelet
6 history
```

CONTROLPLANE

```
15 g drain node01 --ignore-daemonsets
16 g get nodes
17 g uncordon node01
18 g get nodes
19 history
```

Create a Deployment named nginx with 3 replicas. The Pods should use the nginx:1.23.0 image and the name nginx. The Deployment uses the label tier=backend. The Pod template should use the label app=v1.

List the Deployment and ensure that the correct number of replicas is running.

Update the image to nginx:1.23.4.

Verify that the change has been rolled out to all replicas.

Assign the change cause "Pick up patch version" to the revision.

Scale the Deployment to 5 replicas.

Have a look at the Deployment rollout history.

Revert the Deployment to revision 1.

Ensure that the Pods use the image nginx:1.23.0.

(Optional) Discuss: Can you foresee potential issues with a rolling deployment? How do you configure a update process that first kills all existing containers with the current version before it starts containers with the new version?

```
apiVersion : apps/v1
kind       : Deployment
metadata   :
  name     : nginx
  labels   :
```

```
    tier : backend
spec :
  replicas : 3
  selector :
    matchLabels :
      app : v1
  template :
    metadata :
      labels :
        app : v1
    spec :
      containers :
        - name : nginx
          image : nginx:1.23.0
```

- 1 cat > 29.yaml
- 2 kubectl create -f 29.yaml
- 3 kubectl get deploy
- 4 kubectl set image deployment/nginx nginx=nginx:1.23.4
- 5 kubectl rollout history deployment nginx
- 6 kubectl rollout history deployment nginx --revision=1
- 7 kubectl rollout history deployment nginx --revision=2
- 8 kubectl annotate deployment nginx kubernetes.io/change-cause="Pick up patch version"
- 9 kubectl rollout history deployment nginx
- 10 kubectl scale deployment nginx --replicas=5
- 11 kubectl get po -l app=v1
- 12 kubectl get po -o wide
- 13 kubectl get deploy
- 14 kubectl rollout undo deployment/nginx --to-revision=1
- 15 kubectl rollout history deployment nginx

- 16 `kubectl rollout undo deployment/nginx --to-revision=3`
- 17 `kubectl rollout undo deployment/nginx --to-revision=2`
- 18 `kubectl rollout history deployment nginx`
- 19 `history`

1. Create a new Pod from the YAML manifest in the file `pod.yaml`.
2. Check the Pod's status. Do you see any issue?
3. Render the logs of the running container and identify an issue.
4. Shell into the container. Can you verify the issue based on the rendered log message?
5. Suggest solutions that can fix the root cause of the issue.

```
apiVersion: v1
kind: Pod
metadata:
  name: date-recorder
spec:
  containers:
  - name: debian
    image: gcr.io/distroless/nodejs20-debian11
    command: ["/nodejs/bin/node", "-e", "const fs = require
('fs'); let timestamp = Date.now(); fs.writeFile('/root/tmp/s
tartup-marker.txt', timestamp.toString(), err => { if (err) {
console.error(err); } while(true) {} });"]
```

```
cat > 30.yaml
2 kubectl create -f 30.yaml
3 kubectl get po
4 kubectl logs date-recorder
5 kubectl exec -it date-recorder -- /bin/sh
6 kubectl debug -it date-recorder --image=busybox --target=debian --share-
processes

ps

7 history
```

```
apiVersion: v1
kind: Pod
metadata:
  name: date-recorder
spec:
  containers:
    - name: debian
      image: gcr.io/distroless/nodejs20-debian11
      command: ["/nodejs/bin/node", "-e", "const fs = require
('fs'); let timestamp = Date.now(); fs.writeFile('/var/startu
p/startup-marker.txt', timestamp.toString(), err => { if (er
r) { console.error(err); } while(true) {} });"]
      volumeMounts:
        - mountPath: /var/startup
          name: init-volume
  volumes:
    - name: init-volume
      emptyDir: {}
```

1. Create the objects from the YAML manifest `setup.yaml`.
2. Inspect the objects in the namespace `y72`.
3. Create a temporary Pod using the image `busybox:1.36.1` in the namespace `y72`.
The container command should make a `wget` call to the Service `web-app`.
The `wget` will not be able to establish a successful connection to the Service.
4. Identify the root cause for the connection issue and fix it. Verify the correct behavior by repeating the previous step. The `wget` call should return a successful response.

```
apiVersion: v1
kind: Namespace
metadata:
  name: y72
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: y72
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
```

```

template:
  metadata:
    labels:
      app: web-app
  spec:
    containers:
      - image: bmuschko/nodejs-hello-world:1.0.0
        name: web-app
        ports:
          - containerPort: 3000
            protocol: TCP
---
apiVersion: v1
kind: Service
metadata:
  name: web-app
  namespace: y72
spec:
  type: ClusterIP
  selector:
    run: myapp
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3001

```

- 9 cat > 30.yaml
- 10 kubectl create -f 30.yaml
- 11 kubectl get all -n y72
- 12 kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm -n y72 --
wget web-app
- 13 kubectl get endpoints -n y72
- 14 kubectl describe svc web-app -n y72
- 15 kubectl edit svc web-app
- 16 kubectl edit svc web-app -n y72
- 17 kubectl get endpoints -n y72

```
18 kubectl describe svc web-app -n y72
19 kubectl run tmp --image=busybox:1.36.1 --restart=Never -it --rm -n y72 --
wget web-app
20 history
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: y72
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: y72
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - image: bmuschko/nodejs-hello-world:1.0.0
        name: web-app
        ports:
        - containerPort: 3000
          protocol: TCP
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: web-app
  namespace: y72
spec:
  type: ClusterIP
  selector:
    app: web-app
  ports:
    - port: 80
      protocol: TCP
      targetPort: 3000
```

Backup etcd db - and save it to root with the name of the backup "etcd-backup.db"

```
1 kubectl get pods -A | grep etcd
2 kubectl describe pod etcd-controlplane -n kube-system | grep -A20 -i
command
3 ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --
cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt
--key=/etc/kubernetes/pki/etcd/server.key snapshot save etcd-backup.db
4 ls
```

RESTORATION COMMAND

```
etcdctl --data-dir <location> snapshot restore snapshot.db
```

1. Create a new Pod named `hello` with the image `bmuschko/nodejs-hello-world:1.0.0` that exposes the port 3000. Provide the name `nodejs-port` for the container port.
2. Add a Readiness Probe that checks the URL path `/` on the port referenced with the name `nodejs-port` after a 2 seconds delay. You do not have to define the period interval.
3. Add a Liveness Probe that verifies that the app is up and running every 8 seconds by checking the URL path `/` on the port referenced with the name `nodejs-port`. The probe should start with a 5 seconds delay.
4. Shell into container and curl `localhost:3000`. Write down the output. Exit the container.
5. Retrieve the logs from the container. Write down the output.

```
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp : null
  labels:
    run: hello
  name: hello
spec:
  containers:
  - image: bmuschko/nodejs-hello-world:1.0.0
    name: hello
    ports:
      - containerPort: 3000
        name : nodejs-port
```

```

    readinessProbe :
      httpGet :
        path : /
        port : nodejs-port
        initialDelaySeconds : 2

    livenessProbe :
      httpGet :
        path : /
        port : nodejs-port
        initialDelaySeconds : 5
        periodSeconds : 8

    resources: {}
    dnsPolicy: ClusterFirst
    restartPolicy: Never
    status: {}

```

8 kubectl run hello --image=bmuschko/nodejs-hello-world:1.0.0 --port=3000 -o
yaml --dry-run=client --restart=Never > pod.yaml

9 ls

10 kubectl apply -f pod.yaml

11 kubectl exec hello -it -- /bin/sh

12 kubectl get pods

13 kubectl exec hello -it -- /bin/sh

14 kubectl logs pod/hello

15 history

```

# values-dev.yaml
replicaCount: 1
env: development
config:

```

```
  greeting: "Hello from DEV!"
resources:
  requests:
    cpu: "50m"
    memory: "64Mi"
```

```
# values-prod.yaml
replicaCount: 4
env: production
ingress:
  enabled:

true

host: myapp.prod.example.com
config:
  greeting: "Hello from PROD!"
resources:
  requests:
    cpu: "200m"
    memory: "256Mi"
  limits:
    cpu: "500m"
    memory: "512Mi"
```

- 1 helm create my-app
- 2 helm install my-release ./my-app --dry-run --debug
- 3 helm install my-release ./my-app
- 4 kubectl get po
- 5 cat > values-dev.yaml

```
6 cat > values-prod.v.yaml
7 cat > values-prod.yaml
8 helm install my-release ./my-app --namespace prod --create-namespace
9 helm install my-release ./my-app -f values-prod.yaml
10 helm upgrade my-release ./my-app -f values-prod.yaml
11 kubectl get po
12 helm history my-release
13 helm history my-release
14 helm rollback my-release
15 helm uninstall my-release
16 helm repo add bitnami https://charts.bitnami.com/bitnami
17 helm repo add stable https://charts.helm.sh/stable
18 helm repo list
19 helm repo update
20 helm search repo nginx
21 helm install my-nginx bitnami/nginx
22 kubectl get po
23 history
```